Sesion 1: Fundamentos QA Cloud-Native + Gestion avanzada de datos de prueba

Datos sinteticos, anonimizacion GDPR y versionado para ejecucion paralela

QA Cloud-Native - Automatizacion Inteligente con IA Generativa

Que es QA Cloud-Native | ¿Que es GDPR? | ¿Que es anonimizar? | El problema de los datos | Caso 1: Mockaroo

Caso 2: Faker (por codigo) | Caso 3: Anonimizacion | Caso 4: Versionado y paralelo | Enterprise: Data Factory

Que es QA Cloud-Native

El punto de partida de la Certificacion 5

Enfoque

Un QA **cloud-native** no prueba en su maquina y ya. Prueba en entornos que nacen y mueren en la nube, ejecuta en paralelo, integra las pruebas al pipeline CI/CD, y usa **IA generativa** para generar, mantener y reparar casos. Este curso recorre ese camino: hoy arrancamos por el cimiento que sostiene TODO lo demas: **los datos de prueba**.

Por que empezar por los datos

Puedes tener el mejor framework, el pipeline mas veloz y la nube mas cara: si tus datos de prueba son malos, tus resultados son mentira. Datos que se pisan entre si rompen la ejecucion paralela. Datos reales de clientes en un entorno de test son una multa de GDPR esperando a pasar. Hoy resolvemos esos tres problemas: generar datos buenos, anonimizar los reales, y versionarlos para correr en paralelo.

Bloque	Tema	Herramienta
1	Datos sinteticos rapidos y visuales	Mockaroo
2	Datos sinteticos por codigo (repetibles, en CI)	Faker.js + Node
3	Anonimizacion de datos reales + GDPR	Script Node (mask/hash)
4	Versionado y aislamiento para ejecucion paralela	seed/restore + json-server
-	Enterprise: orquestacion de datos	Azure Data Factory (demo/concepto)

¿Que es GDPR?

El marco legal que define como manejamos datos personales en entornos de prueba

Definicion

GDPR son las siglas de **General Data Protection Regulation** (Reglamento General de Proteccion de Datos). Es la ley de privacidad de la Union Europea, vigente desde mayo de 2018. Aunque es europea, **aplica a cualquier empresa del mundo** que maneje datos de ciudadanos europeos. En latinoamerica existen leyes equivalentes con el mismo principio.

Que considera un "dato personal"

> Cualquier dato que identifique, o pueda identificar, a una persona real. En testing, los mas comunes son:

Dato personal	Por que es sensible	Lo que hacemos en testing
Nombre completo	Identifica directamente a la persona	Sustitucion sintetica (Faker)
Email	Permite contactar y rastrear a la persona	Enmascarado: `ju***@***.com`
Documento / cedula	Identificador unico legal	Hash SHA-256 irreversible
Telefono	Permite contacto directo	Enmascarado: `********01`
Fecha de nacimiento exacta	Ayuda a reidentificar junto a otros datos	Generalizacion: solo el año
Direccion IP	Puede revelar ubicacion	Anonimizar o eliminar

> **El problema concreto en testing**
> Copiar la base de datos de produccion a un entorno de pruebas es una **violacion de GDPR**. El entorno de test tiene menos controles de seguridad: menos restricciones de acceso, logs menos monitoreados, posiblemente sin cifrado. Si hay una fuga en ese entorno, la empresa responde legalmente aunque la intencion fuera solo "probar". La solucion no es negarse a usar datos representativos: es **anonimizarlos antes de que salgan de produccion**.

Pais / Region	Ley equivalente a GDPR
Union Europea	GDPR — General Data Protection Regulation (2018)
Colombia	Ley 1581 de 2012 — Habeas Data
Mexico	LFPDPPP (Ley Federal de Proteccion de Datos Personales)
Chile	Ley 19.628 (en proceso de actualizacion)
Argentina	Ley 25.326 de Proteccion de Datos Personales
Brasil	LGPD — Lei Geral de Proteção de Dados (2020)
EE.UU.	No hay ley federal unica; CCPA en California

💡 Para el curso usamos "GDPR" como termino, porque es el estandar internacional de referencia. Si tu empresa opera en Colombia, aplica la Ley 1581. El principio es identico: datos personales en entornos de prueba deben estar anonimizados.

¿Que es anonimizar?

La tecnica que convierte un dato peligroso en un dato util y legal

> **Definicion**
> **Anonimizar** es transformar un dato personal de tal forma que la persona a la que pertenece **ya no pueda ser identificada**, ni directamente ni combinando ese dato con otros. El dato sigue siendo util para hacer pruebas (sigue teniendo el mismo formato, la misma longitud, el mismo tipo), pero la identidad real desaparece.

> **Anonimizar NO es borrar**
> Borrar el email de un usuario hace que el test no pueda correr (el campo es obligatorio). Anonimizar lo reemplaza por algo que *parece* un email real pero no lo es: `ju***@***.com`. El test pasa igual. La persona esta protegida. Esa es la diferencia clave.

Tecnica	Como funciona	Ejemplo: email `juan.garcia@gmail.com`	Reversible
Enmascarado (mask)	Oculta parte del valor dejando el formato visible	`ju***@***.com`	No
Hash SHA-256	Convierte el dato en un codigo fijo e irreversible. Mismo dato = mismo codigo siempre	`5d41402abc4b`	No
Sustitucion sintetica	Reemplaza el dato real por uno falso generado con Faker. Nada del original queda	`wendy.hoeger@example.net`	No
Generalizacion	Reduce la precision del dato para que no identifique a una persona concreta	Fecha `1988-06-14` → solo `1988`	No

La propiedad mas importante: irreversibilidad

Un dato esta bien anonimizado cuando **no hay forma de volver al dato original**. El hash SHA-256 cumple esto para identificadores largos: nadie puede deshacer el hash para obtener el documento original. Eso es lo que GDPR requiere.

Por que el hash sirve para joins aunque sea irreversible

El mismo documento siempre produce el mismo hash: `1093745812` → siempre `9f2a1c4b7e08`. Permite hacer consultas del tipo "dame todos los pedidos del cliente con hash `9f2a1c4b7e08`" sin saber quien es realmente. Los datos se relacionan entre tablas sin exponer la identidad.

⚠️ **Anonimizar mal es igual de peligroso que no anonimizar.** Si sustituyes el nombre pero dejas el email original, la persona sigue siendo identificable. Debe aplicarse a *todos* los campos sensibles a la vez.

Seudonimizacion vs Anonimizacion

La **seudonimizacion** reemplaza el dato por un pseudonimo pero guarda la tabla de correspondencia (reversible con la llave). La **anonimizacion** destruye esa correspondencia: no hay forma de volver. Solo la anonimizacion completa elimina la obligacion de proteger el dato bajo GDPR.

	Seudonimizacion	Anonimizacion
¿Se puede revertir?	Si (con la llave)	No
¿Sigue siendo dato personal?	Si (bajo GDPR)	No
¿Usable en test sin restricciones?	Con controles adicionales	Si
Ejemplo	Token guardado en tabla aparte	Hash sin tabla de correspondencia

💡 En el Ejercicio 3 hacemos **anonimizacion real**: el script no guarda ninguna tabla de correspondencia. Una vez que corres `npm run anonimizar`, no hay forma de recuperar el nombre, email o documento original.

El problema de los datos de prueba

Tres dolores reales que vamos a resolver hoy

Dolor 1: no hay suficientes datos, o son de juguete

"Juan Perez, juan@test.com" repetido 3 veces no prueba nada. Necesitas volumen, variedad y casos de borde (nombres con acentos, emails raros, fechas extremas, campos vacios). Generarlos a mano no escala.

Dolor 2: usar datos reales de produccion

Copiar la base de produccion a un entorno de test es comodo... y peligroso. Nombres, documentos, emails y telefonos reales en un entorno menos protegido violan **GDPR** (y su equivalente local). La solucion no es "no usar datos reales", es **anonimizarlos**.

Dolor 3: la ejecucion paralela se pisa a si misma

Si 5 pruebas corren a la vez y todas crean "el usuario 1", chocan. La ejecucion paralela cloud-native exige datos **aislados y versionados**: cada corrida arranca desde un estado conocido.

⚠️ Tener en cuenta: **dato bueno = realista + suficiente + aislado + legal**

Caso practico 1: Mockaroo (datos sinteticos visuales)

Generar 1000 usuarios realistas en 5 minutos, sin escribir codigo

Que es Mockaroo

Un generador de datos sinteticos en el navegador. Defines columnas y su TIPO (nombre, email, pais, fecha, numero, valor de una lista...) y descarga miles de filas realistas en CSV, JSON, SQL o Excel. El plan gratuito da hasta 1000 filas por descarga: mas que suficiente para practicar.

El escenario

Simulamos una tienda. Necesitamos un dataset de usuarios para probar el registro, el login y los filtros: id, nombre, email, pais, fecha de registro, y un estado (activo/inactivo/pendiente). Lo generamos en Mockaroo y lo bajamos como CSV y como JSON.

🥕 EJERCICIO PRACTICO

Ejercicio 1: generar usuarios en Mockaroo

1. **Entra a `mockaroo.com` e inicia sesion. Veras una tabla con columnas de ejemplo. Borra las que trae por defecto.**
2. **Crea estas columnas y asigna su Type: `id` (Row Number), `nombre` (Full Name), `email` (Email Address), `pais` (Country), `fecha_registro` (Datetime), `estado` (Custom List: activo, inactivo, pendiente).**
3. **En la columna `estado`, usa Custom List y activa "blank %" al 10% para forzar casos de borde (usuarios sin estado). Un buen dataset incluye datos imperfectos.**
4. **Pon `# Rows = 1000`, Format = CSV, y "Download Data". Repite con Format = JSON. Guarda ambos en `proyecto/datos-sinteticos/mockaroo/`.**
5. **Abre el CSV en VS Code. Observa la variedad: acentos, paises distintos, fechas dispersas, algunos `estado` vacios. Eso es un dato REALISTA.**

Definicion de columnas (resumen de lo que configuras en Mockaroo)

```
Field           Type            Opciones
-------------------------------------------------------------
id              Row Number
nombre          Full Name
email           Email Address
pais            Country
fecha_registro  Datetime        formato ISO
estado          Custom List     activo, inactivo, pendiente  (blank 10%)
```

usuarios.csv (primeras filas generadas)

```
id,nombre,email,pais,fecha_registro,estado
1,Maria Gonzalez,mgonzalez@example.com,Colombia,2024-03-11T08:22:00Z,activo
```

```
2,Andres Nino,anino@example.net,Mexico,2023-11-02T14:05:00Z,pendiente
3,Sofia Perez,sperez@example.org,Espana,2024-07-19T09:41:00Z,
4,Luis Ramirez,lramirez@example.com,Argentina,2022-05-30T17:12:00Z,inactivo
```

💡 **Mockaroo tambien exporta como SQL (INSERT statements) y puede exponer un Mock API. Para el curso nos quedamos con CSV/JSON porque los vamos a cargar en json-server y a versionar en Git.**

Caso practico 2: Faker.js (datos sinteticos por codigo)

Cuando necesitas generar datos DENTRO de un pipeline, repetibles y sin limite de filas

Mockaroo vs Faker: cuando cada uno

Mockaroo es genial para generar un dataset rapido a mano. Pero un pipeline CI/CD no abre un navegador ni hace click en "Download". Para eso esta Faker: una libreria que genera los mismos datos realistas por codigo, se ejecuta en la terminal, y con una semilla (seed) produce SIEMPRE el mismo resultado (reproducibilidad, clave en testing).

Preparar (en proyecto/datos-sinteticos)

```
npm init -y
npm i @faker-js/faker
```

generar-usuarios.js

```
import { faker } from '@faker-js/faker';
import fs from 'fs';

// Semilla fija: mismos datos en cada corrida (reproducible en CI)
faker.seed(123);

const estados = ['activo', 'inactivo', 'pendiente'];

function crearUsuario(id) {
  return {
    id,
    nombre: faker.person.fullName(),
    email: faker.internet.email().toLowerCase(),
    pais: faker.location.country(),
    fecha_registro: faker.date.past({ years: 2 }).toISOString(),
    // 10% sin estado -> casos de borde, como en Mockaroo
    estado: Math.random() < 0.1 ? '' : faker.helpers.arrayElement(estados)
  };
}

const usuarios = Array.from({ length: 1000 }, (_, i) => crearUsuario(i + 1));

fs.writeFileSync('faker/usuarios.json', JSON.stringify(usuarios, null, 2));
console.log('Generados ' + usuarios.length + ' usuarios en faker/usuarios.json');
```

Ejecutar

```
node generar-usuarios.js
# Generados 1000 usuarios en faker/usuarios.json
```

La diferencia que importa: reproducibilidad

> Con `faker.seed(123)`, cada vez que corres el script obtienes EXACTAMENTE los mismos 1000 usuarios. Eso es oro en testing: si una prueba falla, puedes regenerar el mismo dataset y reproducir el fallo. Con Mockaroo cada descarga es distinta. Quita el `seed` y veras datos diferentes en cada corrida.

🥕 EJERCICIO PRACTICO

Ejercicio 2: generar datos por codigo

1. Crea la carpeta `faker/`. Escribe `generar-usuarios.js` con el codigo de arriba (o copialo).
2. Corre `node generar-usuarios.js`. Abre `faker/usuarios.json`: 1000 usuarios.
3. Corre el script OTRA VEZ y compara los dos primeros usuarios: son identicos (gracias al seed). Ahora comenta la linea `faker.seed(123)`, corre de nuevo: cambian. Ese es el punto.
4. Reto rapido: agrega un campo `telefono` con `faker.phone.number()` y un `saldo` con `faker.number.int({ min: 0, max: 5000 })`.

💡 Este script es el germen de un paso de pipeline: "antes de las pruebas, genera datos frescos". En la Sesion 5 (CI/CD) lo enchufamos a un job real.

Caso practico 3: Anonimizacion de datos reales (GDPR)

Cuando SI tienes datos reales, no los uses crudos: enmascara y hashea

> **El escenario legal**
>
> El equipo pide "una copia de produccion para reproducir un bug". Esa copia trae nombres, emails, documentos y telefonos REALES. Ponerla en un entorno de test es un incidente de **GDPR**. La solucion profesional: **anonimizar** antes de que salga de produccion. Dos tecnicas base: **enmascarado** (mask) y **hash**.

Tecnica	Que hace	Cuando usarla
Enmascarado (mask)	Oculta parte del dato: `ju***@***.com`	Cuando quieres conservar el formato pero ocultar el valor
Hash	Convierte el dato en un codigo irreversible (SHA-256)	Cuando necesitas que el mismo valor de siempre el mismo codigo (para joins) pero sin poder revertirlo
Sustitucion sintetica	Reemplaza el dato real por uno falso (Faker)	Cuando el valor no importa, solo que sea realista
Generalizacion	Reduce precision: fecha exacta -> solo el año	Para reducir riesgo de reidentificacion

anonimizar.js - convierte datos reales en datos seguros

```
import crypto from 'crypto';
import fs from 'fs';
import { faker } from '@faker-js/faker';

// Entrada: datos "reales" (aqui simulados). En la vida real vendrian de un dump.
const reales = JSON.parse(fs.readFileSync('reales/clientes.json', 'utf-8'));

// 1) Enmascarar email: conserva 2 letras + dominio oculto
```

```
function maskEmail(email) {
  const [user, domain] = email.split('@');
  return user.slice(0, 2) + '***@***.' + domain.split('.').pop();
}

// 2) Hash irreversible del documento (mismo doc -> mismo hash, sin poder revertir)
function hashDoc(doc) {
  return crypto.createHash('sha256').update(String(doc)).digest('hex').slice(0, 12);
}

const anonimos = reales.map(c => ({
  id: c.id,
  nombre: faker.person.fullName(),          // sustitucion sintetica
  email: maskEmail(c.email),                // enmascarado
  documento: hashDoc(c.documento),          // hash
  telefono: c.telefono.replace(/\d(?=\d{2})/g, '*'), // enmascarado
  año_nacimiento: new Date(c.fecha_nacimiento).getFullYear() // generalizacion
}));

fs.writeFileSync('reales/clientes-anonimos.json', JSON.stringify(anonimos, null, 2));
console.log('Anonimizados ' + anonimos.length + ' registros. Ya son seguros para test.');
```

Antes -> Despues (un registro)

```
ANTES (real, NO puede salir de produccion):
{ "nombre": "Carlos Mendez", "email": "carlos.mendez@gmail.com",
  "documento": "1093745812", "telefono": "3145678901", "fecha_nacimiento": "1988-06-14" }

DESPUES (anonimo, seguro para test):
{ "nombre": "Wendy Hoeger", "email": "ca***@***.com",
  "documento": "9f2a1c4b7e08", "telefono": "********01", "año_nacimiento": 1988 }
```

🖍️ EJERCICIO PRACTICO

Ejercicio 3: anonimizar un dump

1. Genera un dump "real" de prueba: reusa Faker para crear `reales/clientes.json` con nombre, email, documento, telefono y fecha_nacimiento (10-20 registros bastan).
2. Escribe y corre `anonimizar.js`. Compara `clientes.json` con `clientes-anonimos.json` lado a lado.
3. Verifica la propiedad clave del hash: corre el script dos veces sobre el MISMO documento y confirma que da el MISMO hash. Eso permite hacer joins sin exponer el dato.
4. Pregunta de cierre: el nombre lo REEMPLAZAMOS (no se puede recuperar), pero el `año_nacimiento` lo GENERALIZAMOS. ¿Por que no basta con enmascarar el nombre?

⚠️ **El hash NO es cifrado reversible: no hay forma de volver al documento original, y eso es exactamente lo que GDPR quiere. Pero cuidado: si el espacio de valores es pequeno (ej. 1-5 estrellas), un hash se puede adivinar por fuerza bruta. Para identificadores como documentos, es adecuado.**

Caso practico 4: Versionado y aislamiento para ejecucion paralela

Que cada corrida arranque desde un estado conocido (patron seed/restore)

El patron seed + restore

El patron funciona asi: se guarda un archivo `usuarios.seed.json` con el estado original del dataset, versionado en Git. Antes de cada corrida de pruebas se copia ese seed sobre el archivo de trabajo (`data/usuarios.json`). Asi la ejecucion es determinista: no importa que corrio antes ni cuantos datos se crearon o borraron, siempre arrancas desde el mismo estado conocido. En cloud-native con ejecucion paralela, esto es obligatorio.

Cargar los datos generados y restaurar (PowerShell)

```
# 1) usa el JSON de Faker/Mockaroo como semilla versionada
Copy-Item faker/usuarios.json seed/usuarios.seed.json -Force

# 2) antes de cada corrida, restaura el estado conocido
Copy-Item seed/usuarios.seed.json data/usuarios.json -Force

# 3) levanta una API sobre esos datos para probar
npx json-server@0.17.4 data/usuarios.json --port 3000
# http://localhost:3000/usuarios
```

Aislamiento por "namespace" de datos (idea para paralelo)

Cuando corres N pruebas en paralelo, cada una debe tocar SUS datos, no los del vecino. Tecnica simple: prefijo por corrida. La prueba A crea usuarios con email `a-runN@test.com`, la prueba B con `b-runN@test.com`. Nunca chocan y la limpieza es trivial (borra por prefijo). En la Sesion 6 lo llevamos a namespaces reales de Kubernetes.

🪛 EJERCICIO PRACTICO

Ejercicio 4: dataset versionado y determinista

1. Crea `seed/usuarios.seed.json` a partir del JSON de Faker. Este es tu estado original, versionado en Git.
2. Restaura a `data/usuarios.json` y levanta `npx json-server data/usuarios.json --port 3000`. Consulta `http://localhost:3000/usuarios`.
3. Crea un usuario con POST (contaminas el dataset). Detén el servicio, RESTAURA con el seed, vuelve a levantar: el usuario extra desaparecio. Estado determinista.
4. Reto: escribe un pequeno script que cree 3 usuarios con prefijos `a-`, `b-`, `c-` (simulando 3 corridas paralelas) y luego borre solo los `b-`. Verifica que a/c siguen ahi.

💡 **Guarda SIEMPRE el seed en Git, nunca el `db.json` de trabajo. El seed es el contrato de datos; el archivo de trabajo es desechable.**

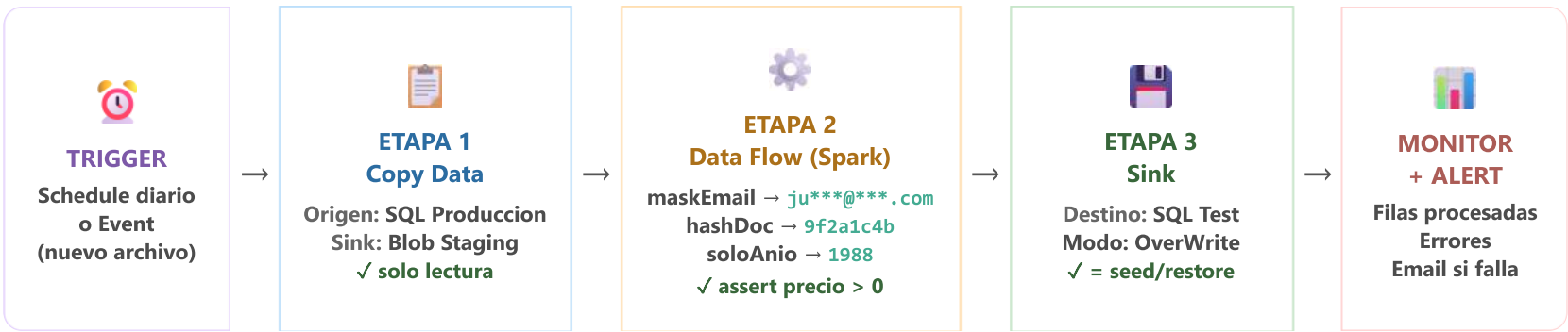
Enterprise: orquestacion con Azure Data Factory

Como se hace esto mismo a escala de empresa — diagrama, ejemplo paso a paso y por que no hacemos el demo en vivo

⚠️ **Por que no hacemos el demo en vivo:** Azure Data Factory requiere suscripcion activa con tarjeta de credito. El cluster Spark de los Data Flows tarda 3-5 minutos en arrancar. Crear y conectar linked services requiere permisos IAM que varían por cuenta. Y el costo, aunque bajo, no es cero. Por eso estudiamos la arquitectura y el ejemplo con detalle — la logica es exactamente la que ya programamos hoy en Node.

Diagrama: pipeline de datos de prueba en ADF

Un pipeline tipico tiene tres actividades encadenadas. Si cualquiera falla, las siguientes no corren y el entorno de test conserva su ultimo estado valido.



🔵 **Etapa 1** = `JSON.parse(fs.readFileSync(...))` 🟠 **Etapa 2** = **funciones** `maskEmail / hashDoc / soloAnio`

🟢 **Etapa 3** = `fs.writeFileSync + seed/restore`

> **Ejemplo: como se configura en el portal de ADF**
>
> **Paso a paso lo que haria un ingeniero de datos al abrir `portal.azure.com` y construir este pipeline. No necesitas hacerlo — es para que entiendas a que corresponde cada concepto cuando lo veas en produccion.**

🪛 EJERCICIO PRACTICO

Como se construye el pipeline en ADF (paso a paso conceptual)

1. **Crear el Data Factory:** portal.azure.com → Create a resource → Data Factory. Nombre: `adf-qa-testdata`, Region: East US. Esto es equivalente a `npm init`: crea el contenedor del proyecto.

2. **Crear Linked Services (las conexiones):** Author → Manage → Linked Services → New. Uno para SQL Server produccion (read-only), uno para Azure Blob Storage (staging), uno para SQL Server test. Equivalente a configurar las cadenas de conexion en un `.env`.

3. **Etapa 1 — Copy Data Activity:** Author → Pipelines → New → arrastrar "Copy data". Source: linked service produccion, query: `SELECT * FROM clientes`. Sink: Blob Storage, path: `staging/clientes-raw.parquet`. Esto es el `generar-dump-real.js` pero conectado a la base real.

4. **Etapa 2 — Data Flow (las transformaciones):** New Data Flow → Source: el Parquet del staging → Derived Column: `maskEmail = concat(left(email,2),'***@***.',right(email,3))`, `hashDoc = sha256(toString(documento))`, `anio_nacimiento = year(fecha_nacimiento)` → Assert: `precio > 0 && cantidad >= 0` → Select: eliminar campos sensibles originales. Esto es exactamente el `2-anonimizar.js` pero con UI visual.

5. **Etapa 3 — Sink:** Sink: linked service SQL Test, tabla: `clientes_anonimos`, Write method: OverWrite (truncate + insert). Equivalente a `npm run restaurar`: cada vez que corre, el entorno de test queda en estado limpio y conocido.

6. **Trigger + Monitor:** Manage → Triggers → New → Schedule: diario a las 02:00 AM. Monitor → Pipeline runs: ver historial de cada corrida, filas procesadas, tiempo. Alert: si el pipeline falla, email automatico al equipo de QA. Esto no tiene equivalente en nuestros scripts — es la pieza que en Node tendriamos que construir a mano.

Nuestro script hoy	Equivalente en Azure Data Factory	Ventaja ADF
`JSON.parse(readFileSync(...))`	Copy Data Activity	Conectores nativos: SQL, Oracle, Salesforce, SAP
`maskEmail / hashDoc` en JS	Data Flow → Derived Column	Corre en Spark: millones de filas en paralelo
`writeFileSync(...)`	Sink Activity	Escribe directo en SQL, Blob, Data Lake
`node 3-anomizar/2-anonimizar.js` a mano	Schedule / Event Trigger	Automatico, sin intervencion humana
Console.log	Monitor + Alerts	Historial, metricas, email si falla
Git diff del script	Source Control integrado	Pipelines versionados y revisados en PR

⚠️ **Por que ADF tiene sentido a partir de cierta escala — y no antes:** el cluster Spark de un Data Flow tarda 3-5 minutos en arrancar aunque proceses 10 filas. El costo minimo es ~$0.25/hora de cluster + $1 por 1000 ejecuciones de actividad. Para un proyecto pequeno

o un equipo de 1-3 personas, `node 3-anomizar/2-anonimizar.js` es gratis, instantaneo y suficiente. ADF tiene sentido cuando superas lo que corre bien en una sola maquina, o cuando necesitas gobierno, auditoria y multiples equipos coordinados.

💡 **Mensaje de cierre del dia: la TECNICA es la misma en pequeno y en grande. Hoy la dominaste con Node — gratis, sin cuenta, en segundos. ADF es esa misma tecnica con Spark distribuido, trigger automatico y auditoria enterprise. Cuando llegues a ese problema, ya sabes exactamente que esta pasando por debajo de los botones.**

A tener en cuenta

- Generar datos sinteticos **visualmente** (Mockaroo) y **por codigo** (Faker, con seed reproducible).
- Anonimizar datos reales para cumplir **GDPR**: mask, hash, sustitucion, generalizacion.
- Versionar con **seed/restore** para ejecucion paralela determinista.
- La logica de ADF es identica a los scripts de hoy — cambia la escala, el gobierno y la automatizacion del trigger.